Software Engineeering

Lab -05

Name:R Muralidharan

SRN:PES2UG24CS387

Section:G

Music_catalog:

Output screenshots:

Initial reports:

```
--------------------------------------------------
>> Issue: [B311:blacklist] Standard pseudo-random generators are not suitable for security/cryptographic purposes.
   Severity: Low   Confidence: High
   CWE: CWE-330 (https://cwe.mitre.org/data/definitions/330.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/blacklists/blacklist_calls.html#b311-random
   Location: .\music_catalog.py:17:11
16          ids = [r[0] for r in conn.execute("SELECT id FROM songs")]
17          return random.choice(ids)
18

--------------------------------------------------
>> Issue: [B301:blacklist] Pickle and modules that wrap it can be unsafe when used to deserialize untrusted data, possible security issue.
   Severity: Medium   Confidence: High
   CWE: CWE-502 (https://cwe.mitre.org/data/definitions/502.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/blacklists/blacklist_calls.html#b301-pickle
   Location: .\music_catalog.py:33:11
32      def loadPlaylist(blob):
33          return pickle.loads(blob)
34

--------------------------------------------------

Code scanned:
        Total lines of code: 35
        Total lines skipped (#nosec): 0
        Total potential issues skipped due to specifically being disabled (e.g., #nosec BXXX): 0

Run metrics:
        Total issues (by severity):
                Undefined: 0
                Low: 2
                Medium: 2
                High: 0
        Total issues (by confidence):
                Undefined: 0
                Low: 0
                Medium: 1
                High: 3
Files skipped (0):
```

```
************* Module music_catalog
music_catalog.py:10:0: C0301: Line too long (109/100) (line-too-long)
music_catalog.py:1:0: C0114: Missing module docstring (missing-module-docstring)
music_catalog.py:9:0: C0116: Missing function or method docstring (missing-function-docstring)
music_catalog.py:9:0: C0103: Function name "addSong" doesn't conform to snake_case naming style (invalid-name)
music_catalog.py:9:0: W0102: Dangerous default value [] as argument (dangerous-default-value)
music_catalog.py:10:17: C0209: Formatting a regular string which could be an f-string (consider-using-f-string)
music_catalog.py:15:0: C0116: Missing function or method docstring (missing-function-docstring)
music_catalog.py:15:0: C0103: Function name "shufflePick" doesn't conform to snake_case naming style (invalid-name)
music_catalog.py:20:0: C0116: Missing function or method docstring (missing-function-docstring)
music_catalog.py:22:4: R1705: Unnecessary "else" after "return", remove the "else" and de-indent the code inside it (no-else-return)
music_catalog.py:28:0: C0116: Missing function or method docstring (missing-function-docstring)
music_catalog.py:28:0: C0103: Function name "savePlaylist" doesn't conform to snake_case naming style (invalid-name)
music_catalog.py:32:0: C0116: Missing function or method docstring (missing-function-docstring)
music_catalog.py:32:0: C0103: Function name "loadPlaylist" doesn't conform to snake_case naming style (invalid-name)
music_catalog.py:36:0: C0116: Missing function or method docstring (missing-function-docstring)
music_catalog.py:36:0: C0103: Function name "totalPlays" doesn't conform to snake_case naming style (invalid-name)
music_catalog.py:39:4: W0702: No exception type(s) specified (bare-except)
music_catalog.py:43:0: C0116: Missing function or method docstring (missing-function-docstring)
music_catalog.py:48:19: C0321: More than one statement on a single line (multiple-statements)

-----------------------------------
Your code has been rated at 4.57/10
```

```
music_catalog.py:6:80: E501 line too long (99 > 79 characters)
music_catalog.py:10:80: E501 line too long (109 > 79 characters)
music_catalog.py:21:80: E501 line too long (86 > 79 characters)
music_catalog.py:39:5: E722 do not use bare 'except'
music_catalog.py:47:5: E741 ambiguous variable name 'l'
music_catalog.py:48:18: E701 multiple statements on one line (colon)
music_catalog.py:49:80: E501 line too long (83 > 79 characters)
```

Updated code:

```python
import json
import random
import sqlite3

conn = sqlite3.connect(":memory:")

conn.execute(
    """
    CREATE TABLE songs (
        id INTEGER PRIMARY KEY,
        title TEXT NOT NULL,
        artist TEXT NOT NULL,
        plays INTEGER NOT NULL
    )
    """
)
def add_song(title, artist, plays=0, playlist=None):
    """Add a song to the database and optionally append it to a playlist."""
    if not isinstance(title, str) or not title.strip():
        raise ValueError("title must be a non-empty string")

    if not isinstance(artist, str) or not artist.strip():
        raise ValueError("artist must be a non-empty string")

    if not isinstance(plays, int) or plays < 0:
        raise ValueError("plays must be a non-negative integer")

    if playlist is None:
        playlist = []

    conn.execute(
        "INSERT INTO songs (title, artist, plays) VALUES (?, ?, ?)",
        (title, artist, plays),
    )

    playlist.append(title)
    return playlist
def shuffle_pick():
    """Return a random song ID, or None if no songs exist."""
    song_ids = [
        row[0] for row in conn.execute("SELECT id FROM songs")
    ]

    if not song_ids:
        return None
```

```
def shuffle_pick():
    """Return a random song ID, or None if no songs exist."""
    song_ids = [
        row[0] for row in conn.execute("SELECT id FROM songs")
    ]

    if not song_ids:
        return None

    return random.choice(song_ids)
def popularity(title):
    """Determine a song's popularity from its play count."""
    row = conn.execute(
        "SELECT plays FROM songs WHERE title = ?",
        (title,),
    ).fetchone()

    if row is None:
        return "unknown"

    return "popular" if row[0] > 100 else "niche"
def save_playlist(playlist):
    """Serialize a playlist of song titles into JSON."""
    if not isinstance(playlist, list) or not all(
        isinstance(title, str) for title in playlist
    ):
        raise ValueError("playlist must be a list of strings")

    return json.dumps(playlist)
def load_playlist(blob):
    """Load and validate a playlist from JSON data."""
    try:
        playlist = json.loads(blob)
    except (json.JSONDecodeError, TypeError) as exc:
        raise ValueError("invalid playlist data") from exc

    if not isinstance(playlist, list) or not all(
        isinstance(title, str) for title in playlist
    ):
        raise ValueError("playlist data must be a JSON list of strings")

    return playlist
```

```
def total_plays():
    """Return the total number of plays across all songs."""
    try:
        result = conn.execute(
            "SELECT SUM(plays) FROM songs"
        ).fetchone()
    except sqlite3.Error:
        return 0

    return result[0] if result[0] is not None else 0
def main():
    """Demonstrate the music catalog functionality."""
    playlist = add_song("Aurora", "Nila", 250)
    add_song("Monsoon", "Kavya", 40, playlist)

    blob = save_playlist(playlist)
    loaded_playlist = load_playlist(blob)

    if len(loaded_playlist) > 1:
        print("multi-song playlist")

    print(
        popularity("Aurora"),
        popularity("Monsoon"),
        shuffle_pick(),
        total_plays(),
    )
if __name__ == "__main__":
    main()
```

Updated reports:

```
************* Module music_catalog
music_catalog.py:124:0: C0304: Final newline missing (missing-final-newline)
music_catalog.py:1:0: C0114: Missing module docstring (missing-module-docstring)

-----------------------------------------------------------------
Your code has been rated at 9.64/10 (previous run: 4.57/10, +5.06)
```

```
Run started:2026-10-16 04:17:26.025509+00:00

Test results:
>> Issue: [B311:blacklist] Standard pseudo-random generators are not suitable for security/cryptographic purposes.
   Severity: Low   Confidence: High
   CWE: CWE-330 (https://cwe.mitre.org/data/definitions/330.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/blacklists/blacklist_calls.html#b311-random
   Location: .\music_catalog.py:51:11
50
51          return random.choice(song_ids)
52

--------------------------------------------------

Code scanned:
        Total lines of code: 90
        Total lines skipped (#nosec): 0
        Total potential issues skipped due to specifically being disabled (e.g., #nosec BXXX): 0

Run metrics:
        Total issues (by severity):
                Undefined: 0
                Low: 1
                Medium: 0
                High: 0
        Total issues (by confidence):
                Undefined: 0
                Low: 0
                Medium: 0
                High: 1
Files skipped (0):
```

```
music_catalog.py:124:11: W292 no newline at end of file
```

Issue	Type	Line(s)	Description	Fix approach
SQL injection	Security	Original line 10	SQL query constructed using string formatting	Parameterized SQL query
Unsafe pickle deserialization	Security	Original line 33	Loading pickle data may execute malicious code	Use JSON for simple playlist data

Mutable default argument	Code quality	Original line 9	Default list can be shared between calls	Use None and initialize a new list
Bare exception	Code quality	Original line 39	Catches all exceptions indiscriminately	Catch sqlite3.Error
Long lines	Style	Multiple	Lines exceed configured limits	Split expressions across lines
Ambiguous variable	Style	Original line 47	Variable l is difficult to understand	Use loaded_playlist